1. PROCEDURE

1. Create a Python Flask application containing three form-security demonstrations: a vulnerable form, input validation, and output encoding.
2. Import the required Flask, regular-expression, and HTML-encoding modules.
3. Create the web interface with separate sections for vulnerable input, validation, and encoding.
4. Run the Flask application locally using 127.0.0.1:5000.
5. Open the application in a web browser.
6. In the Vulnerable Form section, enter the XSS test input <script>alert('XSS Demo')</script> and submit it.
7. Observe that the vulnerable section directly renders the submitted input without output encoding.
8. In the Input Validation section, enter an input containing numbers such as Shaikh123 and click Validate.
9. Observe that the input is rejected because only alphabetic characters and spaces are accepted.
10. In the Output Encoding section, enter the XSS test input and click Encode & Display.
11. Observe that the input is converted into HTML-encoded output instead of being interpreted as HTML.
12. Verify the results displayed in all three sections.

2. SOURCE CODE

File: app.py

```
from flask import Flask, request, render_template_string
import re
import html


app = Flask(__name__)


HTML = """
<!DOCTYPE html>
<html>
<head>
    <title>SSD Lab - Form Security Demonstration</title>
    <style>
        body { font-family: Arial; margin: 30px auto; max-width: 650px;
               background: #f4f6f8; }
        .card { background: white; padding: 20px; margin: 15px 0;
                border-radius: 8px; box-shadow: 0 2px 6px #ccc; }
        input { width: 95%; padding: 10px; margin: 8px 0; }
        button { padding: 9px 15px; }
    </style>
</head>
<body>
    <h1>SSD Lab - Form Security Demonstration</h1>
    <p>Vulnerable Forms, Input Validation and Output Encoding</p>


    <div class="card">
        <h2> 1. Vulnerable Form</h2>
        <p>This form directly renders user input without output encoding.</p>
```

```
    <form method="post">
      <input name="vuln" placeholder="Enter input">
      <button name="action" value="vulnerable">Submit</button>
    </form>
    {% if vuln %}
      <b>Output:</b> {{ vuln|safe }}
    {% endif %}
  </div>

  <div class="card">
    <h2> 2. Input Validation</h2>
    <p>Only alphabetic characters and spaces are accepted.</p>
    <form method="post">
      <input name="name" placeholder="Enter your name">
      <button name="action" value="validate">Validate</button>
    </form>
    {% if validation %}
      <b>Validation Result:</b><br>{{ validation }}
    {% endif %}
  </div>

  <div class="card">
    <h2> 3. Output Encoding</h2>
    <p>User input is HTML-encoded before being displayed.</p>
    <form method="post">
      <input name="encoded" placeholder="Enter input">
      <button name="action" value="encode">Encode & Display</button>
    </form>
    {% if encoded %}
      <b>Encoded Output:</b><br>{{ encoded }}
    {% endif %}
  </div>
</body>
</html>
"""

@app.route("/", methods=["GET", "POST"])
def home():
    vuln = validation = encoded = ""

    if request.method == "POST":
        action = request.form.get("action")

        if action == "vulnerable":
            vuln = request.form.get("vuln", "")

        elif action == "validate":
```

```
        name = request.form.get("name", "")
        if re.fullmatch(r"[A-Za-z ]+", name):
            validation = "✓ Valid input accepted."
        else:
            validation = "✗ Invalid input rejected. Only letters and spaces are allowed."

    elif action == "encode":
        value = request.form.get("encoded", "")
        encoded = html.escape(value)

    return render_template_string(
        HTML, vuln=vuln, validation=validation, encoded=encoded
    )

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000)
```

3. OUTPUT :





